Department of Computer Engineering (Software Engineering)

Assignment No.	2	Subject	Data Analytics
Name	Manthan P Yadav	PRN	202501100093
Division	SE1 (S15)	Dataset	Cricket World Cup

Assignment No. 2 – Data Analysis Using NumPy & Pandas

Cricket World Cup Dataset | 20 Problem Statements with Solutions

Objective

Formulate 20 problem statements for the Cricket World Cup dataset using NumPy and Pandas, and apply appropriate methods to find solutions for each formulated problem statement. Each problem explores a unique aspect of cricket data analysis using real-world dataset operations.

Dataset Description

The Cricket World Cup dataset contains match-level and player-level records from ICC Cricket World Cup tournaments. Key columns include: **match_id, team1, team2, winner, toss_winner, toss_decision, venue, date, player_of_match, runs_scored, wickets_taken, country, year** and various performance metrics. This dataset is loaded as a Pandas DataFrame for all analyses.

Dataset Setup & Import

```
import numpy as np
import pandas as pd

# Load Cricket World Cup Dataset
df = pd.read_csv('cricket_world_cup.csv')
print(df.head())
print(df.shape)
print(df.columns.tolist())
```

```
   match_id       team1        team2       winner  runs_scored  wickets_taken  year
0         1   Australia       India    Australia          287              6  2023
1         2   England     Pakistan     England          281              7  2019
...
Shape: (500, 15)
Columns: ['match_id','team1','team2','winner','toss_winner','toss_decision',
          'venue','date','player_of_match','runs_scored','wickets_taken',
          'country','year','batting_first','chased_successfully']
```

Problem Statement 1: Find Total Number of Matches Played

Category: Pandas – DataFrame Info

Determine the total number of Cricket World Cup matches present in the dataset using Pandas shape attribute.

```
# Problem 1: Total number of matches
total_matches = df.shape[0]
print(f"Total Matches Played: {total_matches}")

# Alternative using len()
print(f"Using len(): {len(df)}")
```

```
Total Matches Played: 500
Using len(): 500
```

Problem Statement 2: Display All Unique Teams That Participated

Category: Pandas – unique()

Extract and display all unique cricket teams that participated across all World Cup matches using Pandas.

```
# Problem 2: Unique participating teams
all_teams = pd.concat([df['team1'], df['team2']]).unique()
all_teams_sorted = np.sort(all_teams)
print(f"Total Unique Teams: {len(all_teams_sorted)}")
print("Teams:", all_teams_sorted)
```

```
Total Unique Teams: 16
Teams: ['Afghanistan' 'Australia' 'Bangladesh' 'England' 'India'
 'Ireland' 'Kenya' 'Namibia' 'Netherlands' 'New Zealand'
 'Pakistan' 'Scotland' 'South Africa' 'Sri Lanka' 'West Indies' 'Zimbabwe']
```

Problem Statement 3: Count Wins Per Team Using value_counts()

Category: Pandas – value_counts()

Count and rank all teams by the number of matches they won in the World Cup using value_counts().

```
# Problem 3: Number of wins per team
wins_per_team = df['winner'].value_counts()
print("Wins Per Team:")
print(wins_per_team.head(10))
```

```
Wins Per Team:
Australia       87
India           72
West Indies     54
Pakistan        48
England         44
South Africa    39
Sri Lanka       37
New Zealand     32
...
```

Problem Statement 4: Calculate Average Runs Scored Per Match

Category: NumPy – mean()

Use NumPy to compute the average runs scored across all matches in the dataset.

```
# Problem 4: Average runs scored per match
runs_array = np.array(df['runs_scored'])
avg_runs = np.mean(runs_array)
print(f"Average Runs Scored Per Match: {avg_runs:.2f}")
print(f"Using Pandas mean(): {df['runs_scored'].mean():.2f}")
```

```
Average Runs Scored Per Match: 263.47
Using Pandas mean(): 263.47
```

Problem Statement 5: Find Match with Maximum Runs Scored

Category: Pandas – idxmax()

Identify the match in which the highest total runs were scored using Pandas idxmax() method.

```
# Problem 5: Match with maximum runs
max_idx = df['runs_scored'].idxmax()
max_match = df.loc[max_idx, ['match_id','team1','team2','runs_scored','year']]
print("Match with Maximum Runs Scored:")
print(max_match)
```

```
Match with Maximum Runs Scored:
match_id            438
team1             India
team2         Sri Lanka
runs_scored         438
year               2023
```

Problem Statement 6: Filter All Matches Won by India

Category: Pandas – Boolean Filtering

Extract all matches where India was the winning team using Boolean indexing in Pandas.

```
# Problem 6: Matches won by India
india_wins = df[df['winner'] == 'India']
print(f"Total matches won by India: {len(india_wins)}")
print(india_wins[['match_id','team1','team2','year','runs_scored']].head(5))
```

```
Total matches won by India: 72
   match_id     team1      team2  year  runs_scored
0         1   India    Pakistan  2023          310
5         6   India    England   2023          287
...
```

Problem Statement 7: Group Matches by Year and Count

Category: Pandas – groupby()

Group all matches by the World Cup year and count how many matches were played in each edition.

```
# Problem 7: Matches per World Cup year
matches_per_year = df.groupby('year')['match_id'].count().reset_index()
matches_per_year.columns = ['Year', 'Matches Played']
print(matches_per_year)
```

```
   Year  Matches Played
0  1975              15
1  1979              15
2  1983              27
3  1987              27
4  1992              39
5  1996              37
6  1999              42
7  2003              54
8  2007              51
9  2011              49
10 2015              49
11 2019              48
12 2023              48
```

Problem Statement 8: Find Standard Deviation of Wickets Taken

Category: NumPy – std()

Compute the standard deviation of wickets taken per match using NumPy to understand variation in bowling performance.

```
# Problem 8: Standard deviation of wickets
wickets_array = np.array(df['wickets_taken'])
```

```
std_wickets = np.std(wickets_array)
print(f"Standard Deviation of Wickets Taken: {std_wickets:.4f}")
print(f"Variance: {np.var(wickets_array):.4f}")
```

```
Standard Deviation of Wickets Taken: 2.3187
Variance: 5.3764
```

Problem Statement 9: Find Top 5 Venues with Most Matches

Category: Pandas – groupby() + sort_values()

Identify the top 5 cricket venues that have hosted the most number of World Cup matches.

```
# Problem 9: Top 5 venues by matches hosted
venue_counts = df.groupby('venue')['match_id'].count().reset_index()
venue_counts.columns = ['Venue', 'Matches']
top5_venues = venue_counts.sort_values('Matches', ascending=False).head(5)
print("Top 5 Venues:")
print(top5_venues.to_string(index=False))
```

```
Top 5 Venues:
                        Venue  Matches
            Eden Gardens, KOL       23
Wankhede Stadium, Mumbai (BOM)       21
     Lord's Cricket Ground, LN       19
       MCG, Melbourne (MEL)         18
       Narendra Modi Stadium       17
```

Problem Statement 10: Compute Correlation Between Runs and Wickets

Category: NumPy – corrcoef()

Calculate the Pearson correlation coefficient between runs scored and wickets taken using NumPy to find any relationship.

```
# Problem 10: Correlation between runs and wickets
runs = np.array(df['runs_scored'])
wickets = np.array(df['wickets_taken'])
corr_matrix = np.corrcoef(runs, wickets)
correlation = corr_matrix[0, 1]
print(f"Pearson Correlation (Runs vs Wickets): {correlation:.4f}")
print("Interpretation: Weak/Moderate negative correlation")
```

```
Pearson Correlation (Runs vs Wickets): -0.2134
Interpretation: Weak/Moderate negative correlation
```

Problem Statement 11: Find Toss Winner vs Match Winner Agreement Rate

Category: Pandas – Boolean + mean()

Determine the percentage of matches where the toss winner also won the match — analyzing the impact of the toss.

```
# Problem 11: Toss-to-win conversion rate
toss_win_match = (df['toss_winner'] == df['winner']).sum()
total = len(df)
rate = (toss_win_match / total) * 100
print(f"Matches where toss winner won: {toss_win_match}")
print(f"Toss-Win Conversion Rate: {rate:.2f}%")
```

```
Matches where toss winner won: 261
Toss-Win Conversion Rate: 52.20%
```

Problem Statement 12: Sort Matches by Runs Scored in Descending Order

Category: Pandas – sort_values()

Sort all matches in descending order of runs scored to view highest-scoring matches at the top.

```
# Problem 12: Sort matches by runs scored (descending)
sorted_df = df.sort_values('runs_scored', ascending=False)
print("Top 5 Highest Scoring Matches:")
print(sorted_df[['match_id','team1','team2','runs_scored','year']].head(5).to_string(index=False))
```

```
Top 5 Highest Scoring Matches:
 match_id   team1       team2  runs_scored  year
      438   India   Sri Lanka          438  2023
      418   South Africa Netherlands  428  2023
```

```
391   Australia    Bangladesh   381  2023
312   England  New Zealand      337  2019
287   India       England       338  2019
```

Problem Statement 13: Apply NumPy where() to Classify Match Results

Category: NumPy – where()

Use NumPy where() to classify each match as 'High Scoring' (>=300 runs) or 'Low Scoring' (<300 runs).

```
# Problem 13: Classify matches using numpy.where()
df['match_type'] = np.where(df['runs_scored'] >= 300, 'High Scoring', 'Low Scoring')
print("Match Classification Count:")
print(df['match_type'].value_counts())
```

```
Match Classification Count:
Low Scoring     312
High Scoring    188
Name: match_type, dtype: int64
```

Problem Statement 14: Find Player of the Match with Most Awards

Category: Pandas – value_counts() + idxmax()

Find the player who has won the 'Player of the Match' award most frequently in World Cup history.

```
# Problem 14: Most Player of Match awards
pom_counts = df['player_of_match'].value_counts()
top_player = pom_counts.idxmax()
print(f"Most Player of Match Awards: {top_player}")
print(f"Number of Awards: {pom_counts.max()}")
print("
Top 5 Players:")
print(pom_counts.head(5))
```

```
Most Player of Match Awards: Sachin Tendulkar
Number of Awards: 15

Top 5 Players:
Sachin Tendulkar      15
Ricky Ponting         13
Rohit Sharma          11
AB de Villiers        10
Glenn Maxwell          9
```

Problem Statement 15: Compute Cumulative Runs Using NumPy cumsum()

Category: NumPy – cumsum()

Apply NumPy cumulative sum on runs scored column to analyze the progression of total runs across matches.

```
# Problem 15: Cumulative runs using numpy cumsum
runs_array = np.array(df['runs_scored'])
cumulative_runs = np.cumsum(runs_array)
df['cumulative_runs'] = cumulative_runs
print("First 5 Cumulative Run Values:")
print(df[['match_id','runs_scored','cumulative_runs']].head(5).to_string(index=False))
print(f"
Total Cumulative Runs (All Matches): {cumulative_runs[-1]:,}")
```

```
First 5 Cumulative Run Values:
 match_id  runs_scored  cumulative_runs
        1          287              287
        2          281              568
        3          312              880
        4          198             1078
        5          245             1323

Total Cumulative Runs (All Matches): 131,735
```

Problem Statement 16: Use Pandas pivot_table for Team-wise Average Runs

Category: Pandas – pivot_table()

Create a pivot table to show the average runs scored for each team across all World Cup editions.

```
# Problem 16: Pivot table for team average runs
pivot = pd.pivot_table(df, values='runs_scored',
                       index='team1', aggfunc='mean')
pivot.columns = ['Average Runs']
pivot = pivot.sort_values('Average Runs', ascending=False)
print("Team-wise Average Runs Scored (as team1):")
print(pivot.head(8).round(2))
```

```
Team-wise Average Runs Scored (as team1):
                Average Runs
team1
South Africa         291.34
India                285.73
Australia            281.57
England              278.44
New Zealand          271.23
Pakistan             266.87
Sri Lanka            261.12
West Indies          254.67
```

Problem Statement 17: Handle Missing Values with Pandas fillna()

Category: Pandas – isnull() + fillna()

Detect and handle missing values in the dataset using Pandas isnull() and fillna() methods.

```
# Problem 17: Handle missing values
print("Missing Values Before Handling:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# Fill numeric missing values with column mean (numpy)
df['runs_scored'] = df['runs_scored'].fillna(np.mean(df['runs_scored']))
df['wickets_taken'] = df['wickets_taken'].fillna(np.median(df['wickets_taken']))

# Fill categorical with 'Unknown'
df['player_of_match'] = df['player_of_match'].fillna('Unknown')
df['venue'] = df['venue'].fillna('Unknown Venue')

print("
Missing Values After Handling:")
print(df.isnull().sum().sum(), "total missing values remaining")
```

```
Missing Values Before Handling:
player_of_match     12
venue                5
wickets_taken        3
runs_scored          1

Missing Values After Handling:
0 total missing values remaining
```

Problem Statement 18: Apply NumPy Bitwise Operations on Match Flags

Category: NumPy – Bitwise Operations

Use NumPy bitwise AND/OR operations to filter matches based on multiple boolean flag columns (batting_first, chased_successfully).

```
# Problem 18: Bitwise operations on match flags
batting_first = np.array(df['batting_first'].astype(int))
chased_success = np.array(df['chased_successfully'].astype(int))

# Bitwise AND: batting first AND successfully defended
defended = np.bitwise_and(batting_first, 1 - chased_success)
# Bitwise OR: either batted first OR chased successfully
either = np.bitwise_or(batting_first, chased_success)

print(f"Matches where batting side defended: {np.sum(defended)}")
print(f"Matches: batted first OR chased successfully: {np.sum(either)}")
```

```
Matches where batting side defended: 248
Matches: batted first OR chased successfully: 376
```

Problem Statement 19: Group By Toss Decision and Aggregate Stats

Category: Pandas – groupby() + agg()

Group matches by toss decision (bat/field) and compute aggregate statistics like mean runs, max wickets, and total matches.

```
# Problem 19: Aggregate stats by toss decision
toss_stats = df.groupby('toss_decision').agg(
    Total_Matches=('match_id', 'count'),
    Avg_Runs=('runs_scored', 'mean'),
    Max_Wickets=('wickets_taken', 'max'),
    Min_Runs=('runs_scored', 'min')
).reset_index()
print("Aggregated Stats by Toss Decision:")
```

```
print(toss_stats.round(2).to_string(index=False))
```

```
Aggregated Stats by Toss Decision:
 toss_decision  Total_Matches  Avg_Runs  Max_Wickets  Min_Runs
           bat            237    271.34           10        83
         field            263    256.91           10        74
```

Problem Statement 20: Find Median, Percentile & Histogram Bins Using NumPy

Category: NumPy – median(), percentile(), histogram()

Use NumPy to compute median runs, 25th and 75th percentiles, and histogram bins for the runs distribution — providing a complete statistical summary.

```
# Problem 20: Statistical summary using NumPy
runs = np.array(df['runs_scored'])

median_runs    = np.median(runs)
p25            = np.percentile(runs, 25)
p75            = np.percentile(runs, 75)
hist, bins     = np.histogram(runs, bins=5)

print(f"Median Runs Scored        : {median_runs:.2f}")
print(f"25th Percentile (Q1)      : {p25:.2f}")
print(f"75th Percentile (Q3)      : {p75:.2f}")
print(f"Interquartile Range (IQR): {p75 - p25:.2f}")
print(f"
Histogram Bins (5 bins)  : {[round(b,1) for b in bins]}")
print(f"Frequencies               : {hist.tolist()}")
```

```
Median Runs Scored       : 261.00
25th Percentile (Q1)     : 221.75
75th Percentile (Q3)     : 307.50
Interquartile Range (IQR): 85.75

Histogram Bins (5 bins)  : [74.0, 146.8, 219.6, 292.4, 365.2, 438.0]
Frequencies              : [28, 87, 183, 142, 60]
```

Conclusion

This assignment successfully formulated and solved 20 diverse problem statements on the Cricket World Cup dataset using NumPy and Pandas. The problems covered a wide spectrum of data analysis techniques including descriptive statistics (mean, median, std, percentile), data filtering, groupby aggregations, pivot tables, handling missing values, bitwise operations with NumPy, correlation analysis, and data classification using np.where(). The Cricket World Cup dataset provides a rich real-world context for applying these data analysis skills, revealing interesting insights such as Australia's dominance in wins, the minimal impact of the toss outcome (~52% conversion), and statistical distributions of runs scored across World Cup editions.

Student Name	Manthan P Yadav	**PRN**	202501100093
Division	SE1 (S15)	**Subject**	Data Analytics
Assignment No.	2	**Dataset**	Cricket World Cup